

Forward and Regressive Arithmetic Goal-Oriented Action Planning

Isaac Hsu (Hsu, Chih-Kang)

Introduction

This article is intended for game developers interested in an AI planning technique known as Goal-Oriented Action Planning (GOAP). Ever since the groundbreaking masterpiece *F.E.A.R.*, this method has spread to other video games, such as *Fallout 3*, *Empire: Total War*, *Middle-earth: Shadow of Mordor*, and *Rise of the Tomb Raider*. Unlike traditional game AI mechanisms including finite state machines and behavior trees, where the decision making is usually hardcoded, GOAP grants AI agents the ability to generate plans for their goals on their own at runtime and decouples decision-making logic from actions and goals. Moreover, the capability of looking ahead makes it stand out from traditional approaches that cannot predict based on previous predictions.

GOAP is a planning architecture for AI to find sequences of actions in order to change game states to given goal states. However, the original GOAP modifies game states only by copying values from actions. If an agent needs to perform an action several times to fulfill a goal (e.g., acquiring sufficient resources for a task), the original method cannot directly describe **partial satisfaction** between the action and the goal. Hence, I would like to propose an extended GOAP with arithmetic operations for expressiveness and flexibility, as this kind of use case could be very common and useful, especially in RPG, strategy, and simulation games.

I will focus on only the planning algorithms of GOAP and will not elaborate on goal selection and plan execution since they are already mentioned in the original GOAP. Additionally, developers may freely devise new processes according to their game design or the features of their game systems and engines. For example, goal selection could be based on user-defined action or goal types, utility scores, or costs of feasible goals, and plan execution could rely on behavior trees, scripts, or event-driven systems.

Related Work

Because this article is based on Dr. Jeff Orkin's GOAP in "*Applying Goal-Oriented Action Planning to Games*", I would like to briefly review it first. In the AI planning algorithm, the problem domain is composed of goals and actions, and solving the problem means finding a series of actions to achieve a given goal.

- **World Property:** Relevant information for AI agents to make decisions, e.g., locations of agents and types of current weapons.
- **World State:** A collection of world properties that an agent holds.
- **Goal:** A desired world state for an agent to achieve. The agent may have multiple goals at the same time and a procedure for goal selection.
- **Action:** Something an agent can do to modify its world state. The changes an action makes can also be described as a world state and are called its **effects**. An action may have optional **preconditions** to meet. For instance, a melee attack requires the target to be at close range. Preconditions can be expressed as a world state as well. For complex conditions, GOAP systems typically allow developers to customize extra procedural preconditions.
- **Plan:** A sequence of actions executed to accomplish a specific goal.

Finding a solution to reach a given goal state from a certain starting state is like searching in a state space tree where the edges represent the actions taken and the nodes represent the world states after those actions on the path from the root are carried out. Therefore, we can use A* to find the shortest path like pathfinding. In fact, a solution is a sequence of actions on a path to produce the goal state.

Laser Example

The laser scenario described by Dr. Orkin in his aforementioned article is utilized to illustrate the search process of GOAP, though the instance is slightly simplified here for brevity.

The goal for the agent is to kill a target, and there is a laser weapon that requires a generator to be switched on. The agent has four actions listed in the following table.

actions		At	Switch	Target
GotoLaser	preconditions			
	effects	laser		
GotoGenerator	preconditions			
	effects	generator		
ActivateGenerator	preconditions	generator		
	effects		on	
FireLaser	preconditions	laser	on	
	effects			dead
start		other	off	alive
goal				dead

Table 1: Actions and world states of the Laser example

We can apply A* to search for a path from the starting state to the goal state, as shown in the following figure.

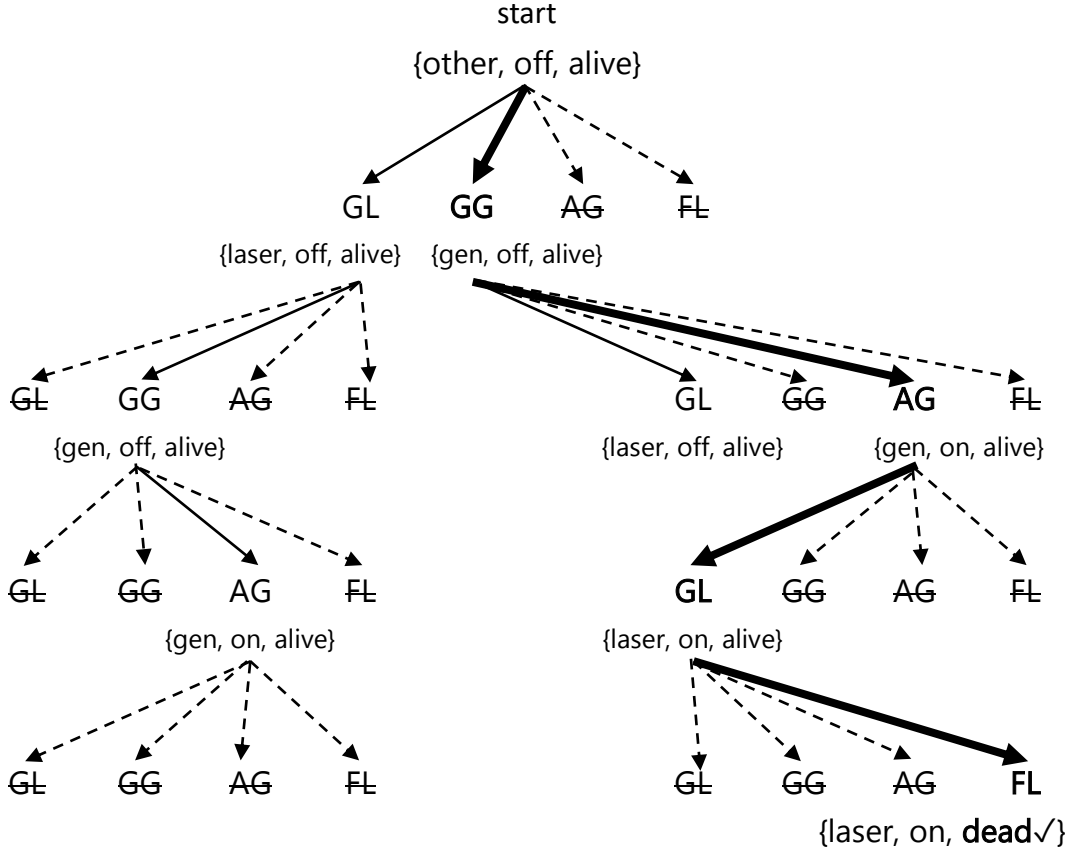


Figure 1: Forward search tree for the Laser example. To simplify the figure, each action can be used only once.

Pathfinding can be done from a start to a goal, or in reverse from a goal to a start, and so can GOAP. In fact, it is generally more efficient to do it backwards because previous possible actions in the backward search are usually fewer than next possible actions in the forward search. Hence, the backward search space is often smaller.

In each node of a backward search tree, we keep track of the current and desired states. When searching backwards from a goal, we initialize the desired state of the root node with the goal state. Also, upon adding new desired properties, we copy the corresponding starting values into the current state. If a property differs in value between the current and desired states, it is deemed unsatisfied. Next, we explore adjacent nodes by finding previous possible actions whose effects satisfy at least one of the unmet desired properties, and update the corresponding current properties to the matching values. Moreover, we add the preconditions of those actions to the desired states as new constraints to be met. Once a node satisfying all desired properties is found, the solution is constructed as the sequence of the actions on the path leading to the root.

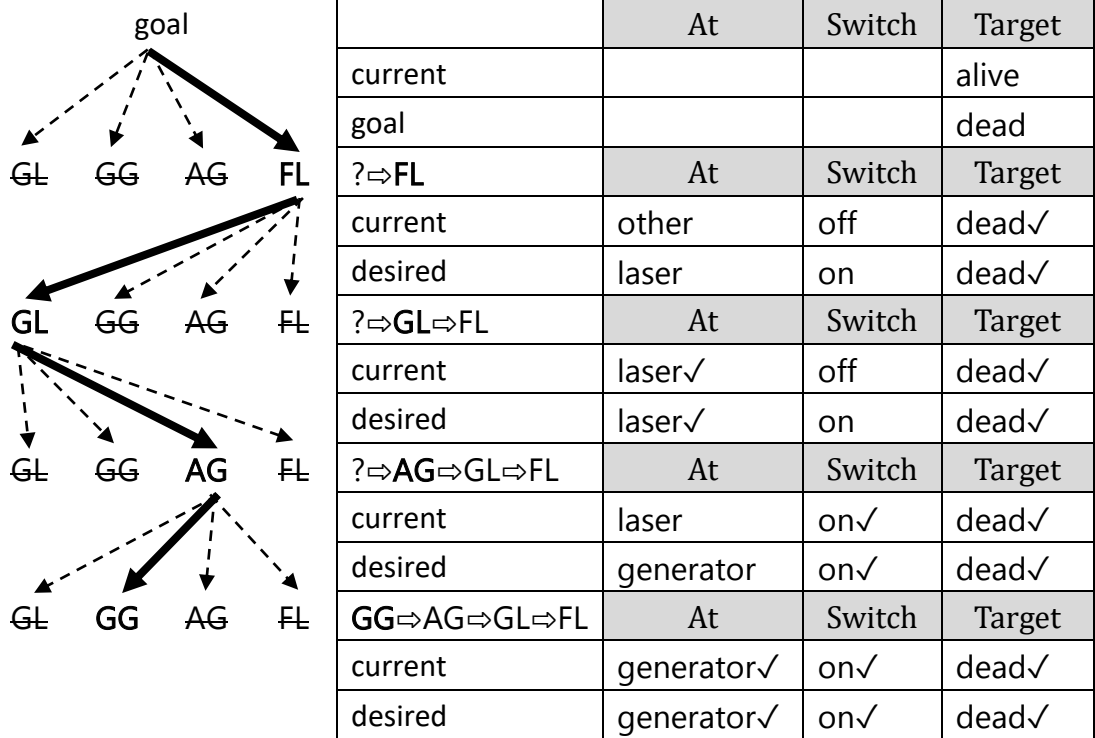


Figure 2: Backward search for the Laser example. Unlike the forward search, there is only one workable backward path from the goal.

Improved Regressive GOAP

In the preceding backward search, for each node, the matching values are copied into the current state and the preconditions are copied into the desired state. However, this approach requires that there is no inconsistency between the preconditions and the effects of an action.

To demonstrate this, we can add a Switch-off precondition to ActivateGenerator in the Laser example and see the result. The modified action will set the current value of Switch to on but the desired value to off, which remains unsatisfied and fails the planning process. On the contrary, the forward search works properly with the same extra precondition. Ideally the backward search should be a more performant replacement for the forward search without additional limitations.

To fix this issue, we can remove satisfied properties from the desired states and instead check if the starting state satisfies the desired state of the current least-cost node, thereby determining whether a solution is found. In addition, when searching for previous feasible actions, we must filter out those whose effects or preconditions conflict with unsatisfied desired properties. Furthermore, the current states of the nodes are no longer needed and can be deleted, which reduces memory usage.

modified action		At	Switch	Target
ActivateGenerator'	preconditions	generator	off	
	effects		on	
		At	Switch	Target
	current			alive
	goal			dead
	$? \Rightarrow FL$	At	Switch	Target
	current	other	off	dead✓
	desired	laser	on	dead✓
	$? \Rightarrow GL \Rightarrow FL$	At	Switch	Target
	current	laser✓	off	dead✓
	desired	laser✓	on	dead✓
	$? \Rightarrow AG' \Rightarrow GL \Rightarrow FL$	At	Switch	Target
	current	laser	onX	dead✓
	desired	generator	offX	dead✓

Figure 3: Backward search for the Laser example with an extra precondition of Switch-off

		At	Switch	Target
	start	other	off✓	alive
	goal			dead
	$? \Rightarrow FL$	At	Switch	Target
	desired	laser	on	dead
	$? \Rightarrow GL \Rightarrow FL$	At	Switch	Target
	desired	laser	on	
	$? \Rightarrow AG' \Rightarrow GL \Rightarrow FL$	At	Switch	Target
	desired	generator	on → off	
	$GG \Rightarrow AG' \Rightarrow GL \Rightarrow FL$	At	Switch	Target
	desired	generator	off✓	

Figure 4: Improved regressive search for the Laser example with an extra precondition of Switch-off

In the forward search, the world states of the nodes are the states modified by the actions on the path from the root. However, what do the world states of the nodes mean in the regressive search? To grasp it, the key idea is that one step in the regressive GOAP means to **reverse** an action and **retrodict** the previous conditions. For example, when we search backwards from the goal in the Laser example, the

only feasible action for the dead Target is FireLaser. Being at the laser and Switch-on are the previous desired constraints when FireLaser is reversed.

Workarounds without Arithmetic Support

In the Laser example, you may have noticed that the planning algorithm is not limited to only Boolean values since the world property of At is actually an enumeration of three possible values: other, laser, or generator. In fact, we can use world properties of any cloneable and comparable types, such as floating-point numbers, strings and game object handles as needed. Nevertheless, in the original GOAP, effects can only copy values rather than transform them, even for simple increments. Conceptually, this means we cannot directly describe a goal that requires repeating an action to achieve it without some tricks.

Suppose in a turn-based simulation game an AI agent wants to buy a house that costs \$1000. It can obtain money by working for others (+\$100), running a shop (+\$300), or taking out a loan (+\$700). Moreover, running a shop and taking out a loan both require a non-negative credit score, and a loan will greatly decrease its credit (-70) so it is only possible once in a while. A small amount of credit score can be gained by working (+10). The agent can work ten times or run a shop four times to get enough money for a house. Obviously, the fastest solution is to run a shop on the first turn and then take out a loan, but not in reverse order due to the credit limit on the loan.

One way of adapting the preceding algorithm for this instance is to define multiple Boolean world properties that represent different thresholds, e.g., whether its money is more than 100, 200, ..., and 1000, respectively, so that the preconditions and the effects can be formulated. Nonetheless, this workaround may not scale well in fine-grained cases.

An alternative is to define a Boolean world property to indicate whether the money is enough for the house purchase. Next, we subclass the world states to add extra numeric fields for balance and credit score, and subclass the actions to add procedural preconditions for balance and credit checks. Although the balance value is hidden from the planner, we can override the cost functions to make A* prefer to get a loan. Nevertheless, none of the three actions can earn \$1000 at once, so we must force the Boolean effect of the actions to be true for the planner to work. But when the plan is executed, it will fail to purchase a house on the first turn and trigger replanning on the next turn. Eventually it will earn enough money after several retries.

actions		EnoughMoney	HasHouse	(Balance)	(Credit)
BuyHouse	preconditions	true		≥ 1000	
	effects		true	-1000	
Work	preconditions				
	effects	true		+100	+10
RunShop	preconditions				≥ 0
	effects	true		+300	
GetLoan	preconditions				≥ 0
	effects	true		+700	-70
start		false	false	10	0
goal			true		

Table 2: Actions and world states of the House example with additional numeric fields hidden from the planner

However, since the planning algorithm still cannot see all the information with this trick, it may result in locally optimal solutions instead of globally optimal one. For instance, it could result in an initial plan of getting a loan followed by three work plans, which is not the best solution. Consequently, it would be better if we could support numeric world properties with arithmetic operations, such as addition and subtraction in GOAP for this kind of need.

Forward Arithmetic GOAP

Regarding operations on world properties, the original GOAP uses only **equality** comparators for goals and preconditions and **assignment** operators for effects. To expand the original method, we can support the **four arithmetic operators** for effects, as well as the two **inclusive comparators** ($\leq$ and $\geq$) for goals and preconditions. The latter two can express ranges of desired values instead of just single values. They are often more useful than equality comparators when used with arithmetic operations, since values that exceed goals still count.

	comparators for preconditions	operators for effects	
general properties	$==$	$=$	
numeric properties	$\leq$	$+$	$-$
	$\geq$	$\times$	$\div$

Table 3: Comparators and operators for general and numeric world properties

House Example using Forward Arithmetic GOAP

Using the arithmetic operators and the inclusive comparators, we can redefine the House example in the following table.

actions		House	Balance	Credit
BuyHouse	preconditions		≥ 1000	
	effects	+1	-1000	
Work	preconditions			
	effects		+100	+10
RunShop	preconditions			≥ 0
	effects		+300	
GetLoan	preconditions			≥ 0
	effects		+700	-70
start		0	10	0
goal		≥ 1		

Table 4: Actions and world states of the House example using the arithmetic operations

Because all the conditions can be exposed to the algorithm now, the planner knows that RunShop should be done before GetLoan and can find the best solution.

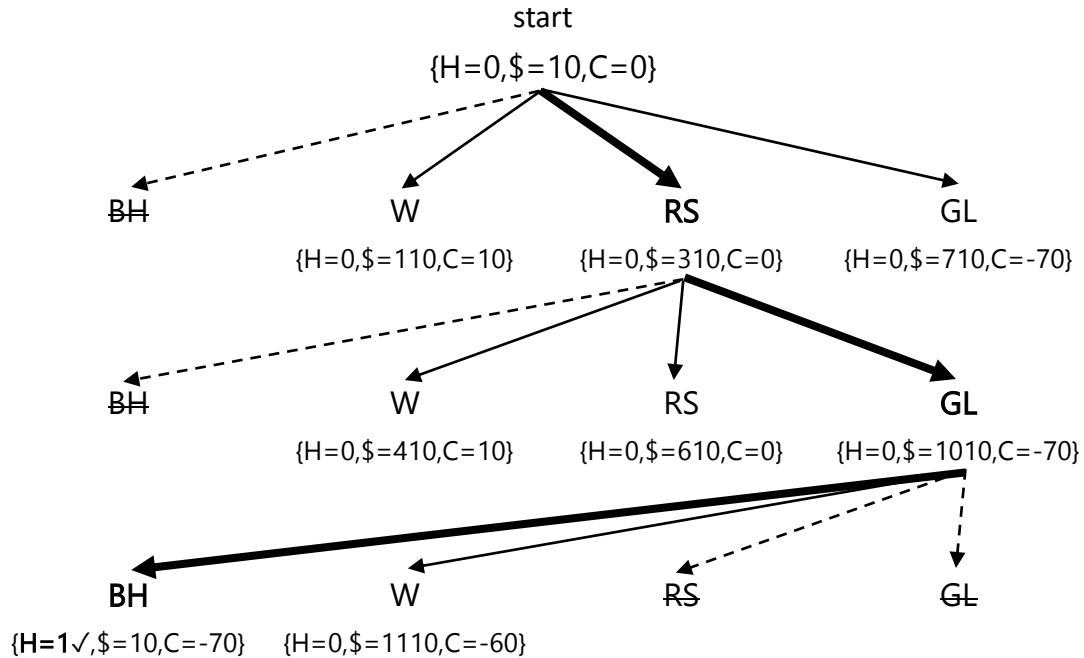


Figure 5: Partial forward search tree for the House example using the arithmetic operations. Due to limited horizontal space, only the sibling branches of the shortest path are shown.

Regressive Arithmetic GOAP

Regressive arithmetic GOAP is similar to the aforementioned improved regressive algorithm. It searches backwards from a goal, reverses possible actions, retrodicts previous desired states, and checks whether the starting state satisfies the current desired state to determine if a solution is found. Moreover, we need to support the two inclusive comparators and the four arithmetic operators.

Addition

To reverse the effects of addition, we can generalize the problem as the following double inequality and solve it:

$$a_1 \leq x + n \leq a_2 \Rightarrow a_1 - n \leq x \leq a_2 - n ,$$

where x denotes the world property in question, $+n$ is the effect, and a_1 and a_2 are the lower and upper bounds of the desired constraints, respectively, which could be $-\infty$ or $+\infty$ if the constraint is unbounded.

For instance, when reversing the BuyHouse action from the goal state in the House example, we can deduce the previous desired constraints as follows:

$$1 \leq \text{House} + 1 \leq \infty \Rightarrow 0 \leq \text{House} \leq \infty ,$$

which means that no House is required before BuyHouse.

Subtraction

Subtraction can be converted to addition, as shown below, and solved like addition.

$$x - n \equiv x + (-n)$$

Multiplication

Also, we can formulate the problem of multiplication and solve it below.

$$a_1 \leq x \times n \leq a_2 \Rightarrow \begin{cases} a_1/n \leq x \leq a_2/n & , n > 0 \\ a_1/n \geq x \geq a_2/n & , n < 0 \\ x == \emptyset & , n == 0 \wedge 0 \notin [a_1, a_2] \\ -\infty \leq x \leq \infty & , n == 0 \wedge 0 \in [a_1, a_2] \end{cases}$$

In the last case, because any value times zero equals zero, x can be any number if the multiplicand is zero and the desired value could be zero. By contrast, there is no solution to the third case, where the multiplicand is zero but the desired constraints exclude zero.

Division

Like subtraction, division can be converted to multiplication by the following formula:

$$x \div n \equiv x \times (1/n).$$

However, in the event of a division by zero, the effect is invalid and the action is infeasible, as it is mathematically undefined.

House Example using Regressive Arithmetic GOAP

The regressive search process for the House example is shown in the figure below.

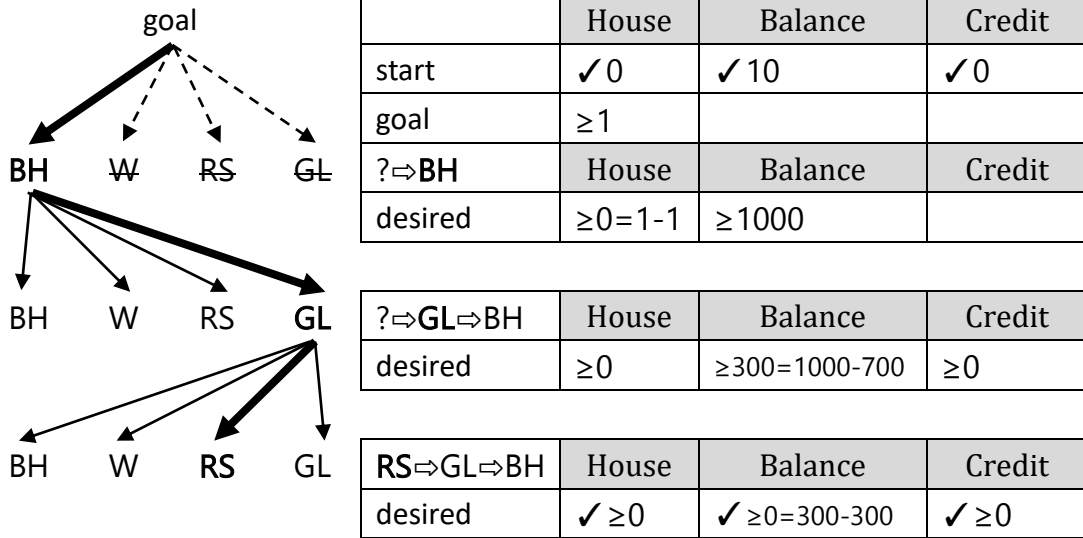


Figure 6: Regressive search for the House example using the arithmetic operations

You may notice that, unlike the original GOAP, reversing the effects applied to desired constraints results in ranges of possible values instead of single values. In addition, when adding preconditions to existing desired properties, rather than simply overwriting them, we compute the common solutions of the existing constraints and the new conditions, which are the intersections of their ranges (see Table 7).

Range Limits on Numeric World Properties

In practice, it may be useful to put range limits on world properties in some cases. Assuming an agent has two actions, Work and Rest, and wants to earn a certain amount of money, we can apply the forward GOAP as shown in the following table.

forward search		Money	Fatigue
Work	preconditions		≤ 0
	effects	+5	+2
Rest	preconditions		≥ 1
	effects		-3
start		0	4
goal		✓ ≥ 8	
R		0	$1 = 4 - 3$
$R \Rightarrow R$		0	-2 = 1 - 3
$R \Rightarrow R \Rightarrow W$		$5 = 0 + 5$	$0 = -2 + 2$
$R \Rightarrow R \Rightarrow W \Rightarrow W$		✓ $10 = 5 + 5$	$2 = 0 + 2$

Table 5: Actions, world states, and forward search process for the Range example without range limits

Nevertheless, Fatigue actually cannot be negative in the game world. Although a precondition is already applied to Fatigue of Rest, it does not restrict the outcome of the action, because it is a precondition instead of a postcondition. If we put range limits on the numeric properties and clamp the results after the effects are applied, we can avoid out-of-range values and find a correct plan as illustrated below.

forward search		Money	Fatigue
range limits		≥ 0	≥ 0
Work	preconditions		≤ 0
	effects	+5	+2
Rest	preconditions		≥ 1
	effects		-3
start		0	4
goal		$\checkmark \geq 8$	
R		0	$1 = \text{clamp}(4-3, [0, \infty))$
$R \Rightarrow R$		0	$0 = \text{clamp}(1-3, [0, \infty))$
$R \Rightarrow R \Rightarrow W$		$5 = \text{clamp}(0+5, [0, \infty))$	$2 = \text{clamp}(0+2, [0, \infty))$
$R \Rightarrow R \Rightarrow W \Rightarrow R$		5	$0 = \text{clamp}(2-3, [0, \infty))$
$R \Rightarrow R \Rightarrow W \Rightarrow R \Rightarrow W$		$\checkmark 10 = \text{clamp}(5+5, [0, \infty))$	$2 = \text{clamp}(0+2, [0, \infty))$

Table 6: Actions, world states, and forward search process for the Range example with non-negative range limits

Regressive Arithmetic GOAP with Range Limits

Implementing range limits for the forward GOAP is straightforward. By contrast, to support them in the regression planning, we must undo the additional clamping of range limits before the arithmetic operations. This is because the regressive search executes operations in reverse order.

If a range limit does not overlap with the current desired constraints, there is no solution to the previous desired constraints. Otherwise, within their intersection, we can deduce solutions by reversing the arithmetic operations as described before. Moreover, if the lowest range limit lies within the desired interval, the unclamped desired minimum is unbounded. This is because even negative infinity gets clamped to the lowest limit, and the problem can still be solved. This works similarly for the desired maximum. We can formulate the unclamping process as follows:

$$a_1 \leq \text{clamp}(x * n, [r_1, r_2]) \leq a_2 \Rightarrow \begin{cases} x == \emptyset & , [r_1, r_2] \cap [a_1, a_2] == \emptyset \\ a_1' \leq x * n \leq a_2' & , [r_1, r_2] \cap [a_1, a_2] \neq \emptyset \end{cases} ,$$

where $*$ can be $+$, $-$, $\times$, or $\div$ and r_1 and r_2 are the bounds of a range.

$$a_1' = \begin{cases} a_1, & r_1 \notin [a_1, a_2] \\ -\infty, & r_1 \in [a_1, a_2] \end{cases} \quad a_2' = \begin{cases} a_2, & r_2 \notin [a_1, a_2] \\ +\infty, & r_2 \in [a_1, a_2] \end{cases}$$

After the unclamping and reversing operations, the resultant desired states should be checked against the range limits owing to the fact that numeric properties are always constrained within their ranges by definition. If there is no intersection between the result and the range, the path is deemed infeasible (see the second path in Table 7).

regressive search		Money	Fatigue
range limits		$[0, \infty]$	$[0, \infty]$
Work	preconditions		$[-\infty, 0]$
	effects	+5	+2
Rest	preconditions		$[1, \infty]$
	effects		-3
start		0✓	4✓
goal		$[8, \infty]$	
$? \Rightarrow W$		$\text{unclamp}([8, \infty], [0, \infty]) = [8, \infty]$ $[8, \infty] - 5 \cap [0, \infty] = [3, \infty]$	$[-\infty, 0] \cap [0, \infty] = 0$
$? \Rightarrow W \Rightarrow W$		$\text{unclamp}([3, \infty], [0, \infty]) = [3, \infty]$ $[3, \infty] - 5 \cap [0, \infty] = [0, \infty]$	$\text{unclamp}([0, 0], [0, \infty]) = [-\infty, 0]$ $[-\infty, 0] - 2 \cap [-\infty, 0] \cap [0, \infty] = \emptyset \times$
$? \Rightarrow R \Rightarrow W$		$[3, \infty]$	$\text{unclamp}([0, 0], [0, \infty]) = [-\infty, 0]$ $[-\infty, 0] + 3 \cap [1, \infty] \cap [0, \infty] = [1, 3]$
$? \Rightarrow W \Rightarrow R \Rightarrow W$		$\text{unclamp}([3, \infty], [0, \infty]) = [3, \infty]$ $[3, \infty] - 5 \cap [0, \infty] = [0, \infty]$	$\text{unclamp}([1, 3], [0, \infty]) = [1, 3]$ $[1, 3] - 2 \cap [-\infty, 0] \cap [0, \infty] = 0$
$? \Rightarrow W \Rightarrow W \Rightarrow R \Rightarrow W$		$\text{unclamp}([0, \infty], [0, \infty]) = [-\infty, \infty]$ $[-\infty, \infty] - 5 \cap [0, \infty] = [0, \infty]$	$\text{unclamp}([0, 0], [0, \infty]) = [-\infty, 0]$ $[-\infty, 0] - 2 \cap [-\infty, 0] \cap [0, \infty] = \emptyset \times$
$? \Rightarrow R \Rightarrow W \Rightarrow R \Rightarrow W$		$[0, \infty]$	$\text{unclamp}([0, 0], [0, \infty]) = [-\infty, 0]$ $[-\infty, 0] + 3 \cap [1, \infty] \cap [0, \infty] = [1, 3]$
$R \Rightarrow R \Rightarrow W \Rightarrow R \Rightarrow W$		$[0, \infty] \checkmark$	$\text{unclamp}([1, 3], [0, \infty]) = [1, 3]$ $[1, 3] + 3 \cap [1, \infty] \cap [0, \infty] = [4, 6] \checkmark$

Table 7: Actions, world states, and regressive search process for the Range example with range limits. The retrodiction of the desired constraints is formulated as interval operations. The range limits are in italics and the constraints from the preconditions are underlined.

Implementation Considerations

In the previous example, reversing the arithmetic operations can be expressed as operations on intervals, so numeric properties in desired states can in fact be described as intervals. Boolean and enumeration properties can also be represented as degenerate intervals. Nonetheless, not all results of reversing numeric operations and functions can be put into a single interval.

In the original GOAP, the heuristic is the number of unsatisfied desired properties. Regarding heuristics for numeric properties, since their constraints can be formulated as intervals, we could calculate and sum interval distances so that A* would prefer effects closer to desired world states. The distance between two intervals can be computed by the following formula.

$$d([a_1, a_2], [b_1, b_2]) = \begin{cases} 0 & , [a_1, a_2] \cap [b_1, b_2] \neq \emptyset \\ \min(|a_1 - b_2|, |a_2 - b_1|) & , [a_1, a_2] \cap [b_1, b_2] = \emptyset \end{cases}$$

However, concerning numeric properties with wide ranges, the heuristic may overestimate the actual costs. To make it admissible, we could lower the heuristic values and/or raise the action costs to ensure that the heuristic is always no greater than the actual costs. One way of doing this is to scale down the heuristic so that it is no more than 1, and additionally to increase the costs of actions by the number of world properties.

In each iteration of A* in GOAP, we need to find feasible actions for the least-cost node in order to explore its adjacent nodes. In the regressive search, we can loop through the actions and check if their effects are desired. Alternatively, we can build a lookup table first and then look up desired properties for candidate actions. Moreover, for world properties with only fixed-value effects, we can build a constant effect table from the actions to identify states with unsatisfiable desired values missing from both the table and the starting state. For numeric properties, we can build an effect direction table to indicate whether the properties can be increased, decreased, or both by these actions. Then, we can detect infeasible desired constraints by seeing if the starting state is unachievable because the required effect directions are not listed in the table.

Although A* pathfinding algorithms usually build closed sets to avoid backtracking, they may not be useful for GOAP. Exploring adjacent nodes using actions already on the path does not mean revisiting the same nodes, but rather repeating the actions. The results could be different, and it is necessary in cases of partial satisfaction, where the actions may need to be performed more than once to satisfy the desired world states. On the other hand, it could be helpful to put limits on the execution times if some actions are permitted only once or at certain times. This may also reduce the search space and expedite the search process.

When there are actions with procedural preconditions on a backward path, it may be necessary to check these preconditions in the forward manner to make sure that they are met as well.

As subtraction is the inverse of addition and division is the inverse of multiplication, we can implement just one for each pair and convert the other to its counterpart.

It could be advantageous to allow the customization of world states, goals, and actions. Imagine that an agent uses GOAP to secure guns and ammunition. It can store its planned locations in subclasses and override A* cost and heuristic functions for the additional costs based on the distance between the planned locations so that it can find the shortest route among several nearby candidates.

Conclusion

GOAP is a great AI technique that empowers agents to map out action plans at runtime to deal with dynamic environments, and enables developers to add behavioral diversity without exponential complexity. By integrating the arithmetic operations and range limits for numeric world properties, we can enhance the applicability and capability of GOAP. With the proposed regressive search and lookup tables, we can save space in the search trees and improve search efficiency.

Furthermore, this regressive algorithm is extensible because we can support more mathematical operations by deriving their solutions. A simple case, such as the solution to Boolean negation, can be deduced as follows: $\neg x == B \Rightarrow x == \neg B$, where B can be true or false.

Reference

Orkin, J. (2003) *Applying Goal-Oriented Action Planning to Games*

Links of Example Code

The C++ code for the examples mentioned is available at the links below.

- Laser example
<https://github.com/lsatin/GOAPPrototypes/blob/main/GOAP/LaserExample/LaserExample.cpp>
- House example
<https://github.com/lsatin/GOAPPrototypes/blob/main/ArithGOAP/HouseExample/HouseExample.cpp>
- Range example
<https://github.com/lsatin/GOAPPrototypes/blob/main/ArithGOAP/RangeExample/RangeExample.cpp>